
MSML 605

Concurrency

Thanks to Manoj Franklin for the slides' content

Code examples modified by Mohammad Nayeem Teli

Machine Learning Hardware

- CPUs (Conventional)
- GPUs
- DSPs
- FPGAs
- Other accelerators



CPU_s

- General-purpose hardware
- An OS can run multiple processes at the same time.
- By default Python interpreter executes instructions serially
- The size of the datasets has increased
- The algorithms are more complex and need to process more, hence the need for multi-processing

Interpreting a Python Program

English: Display the sum of A times B plus C.

Python:

```
print (A * B + C)
```

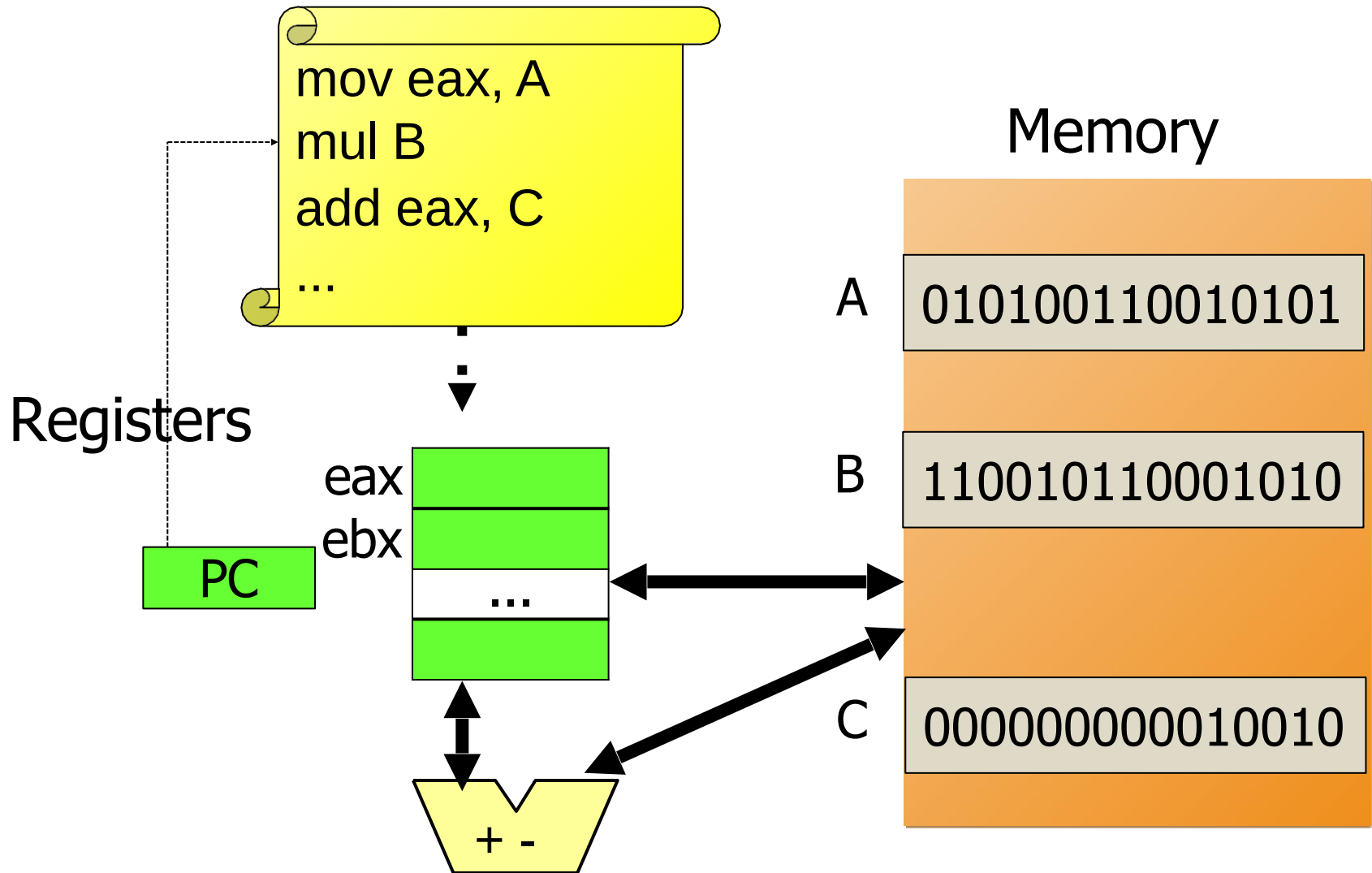
Assembly Language:

```
mov  eax,A  
mul  B  
add  eax,C  
call WriteInt
```

Intel Machine Language:

```
A1  00  00  00  00  00  
F7  25  00  00  00  04  
03  05  00  00  00  08  
E8  00  50  00  00
```

Hardware Model with CPU

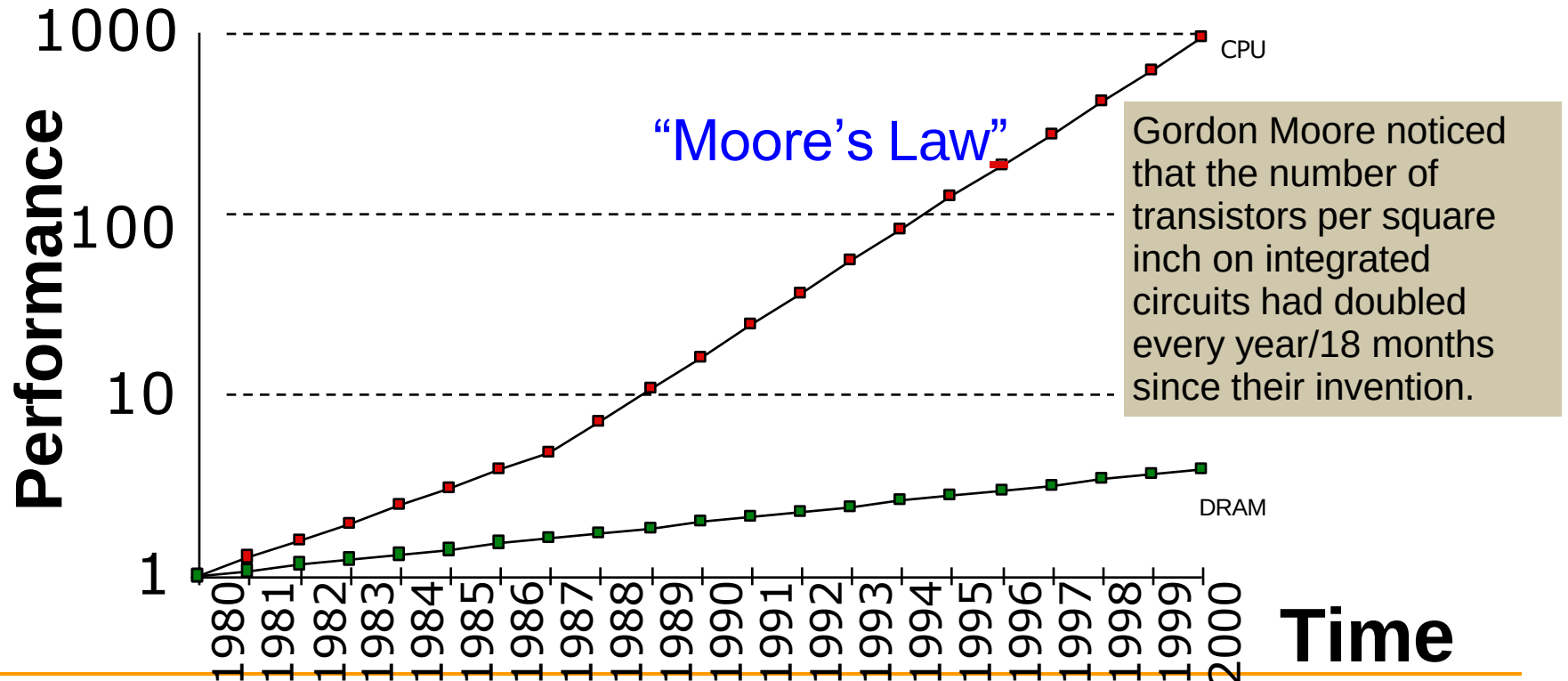


Processor-Memory Performance Gap

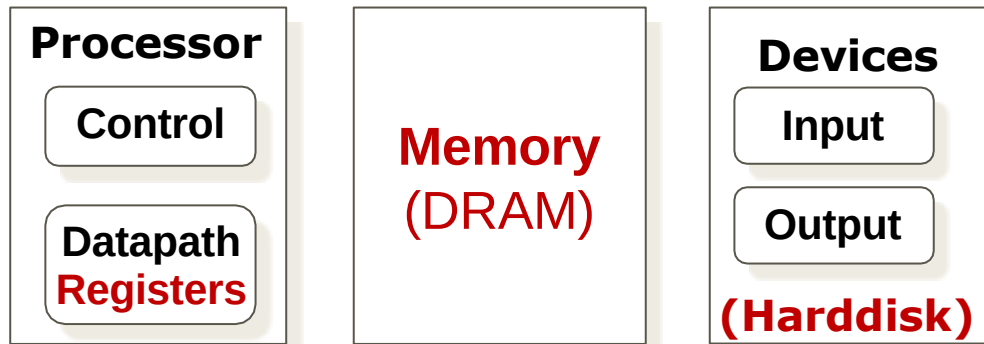
Memory Wall:

1GHz Processor → 1 ns per clock cycle

50ns for DRAM access → 50 processor clock cycles per memory access!



Speed vs Quantity



	Capacity	Latency	Cost/GB
Register	100s Bytes	20 ps	\$\$\$\$
SRAM	100s KB	0.5-5 ns	\$\$\$
DRAM	100s MB	50-70 ns	\$
Hard Disk	100s GB	5-20 ms	Cents
Ideal	100 GB	1 ns	Cheap

Best of Both Worlds

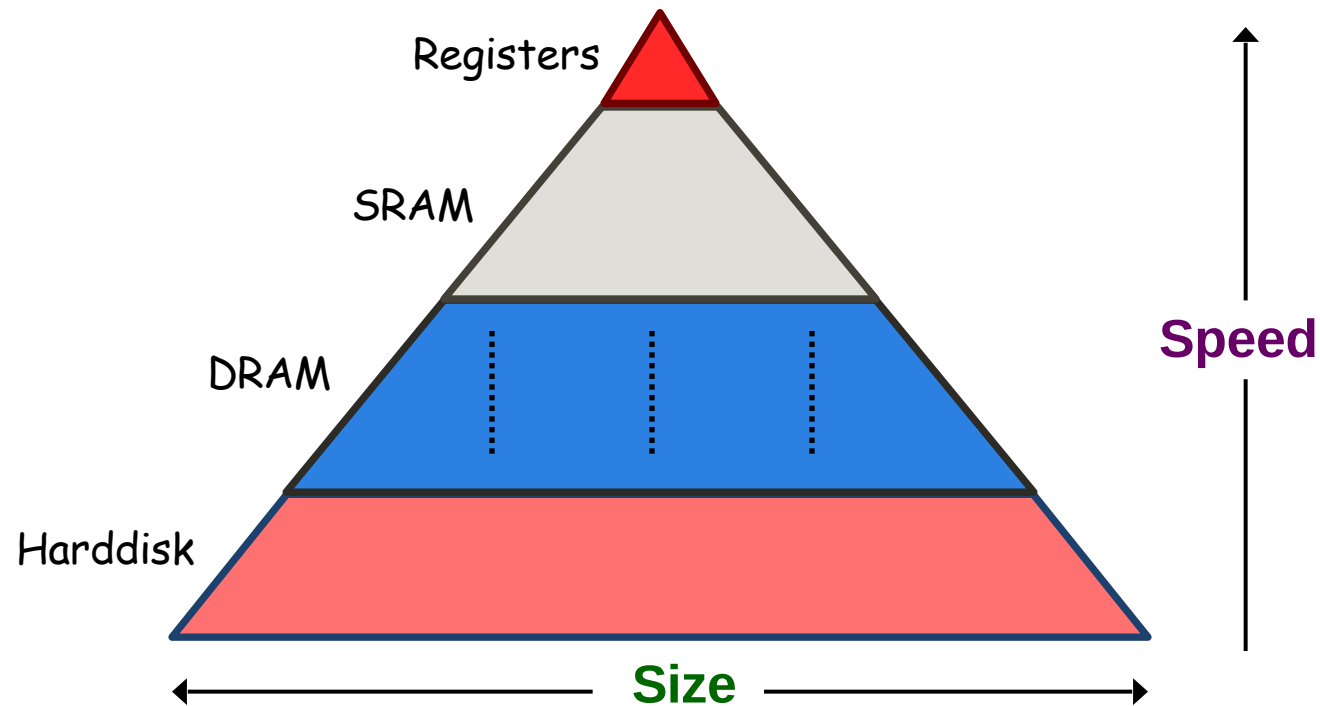
- What we want:
 - A **BIG** and **FAST** memory
 - Memory system should perform like 1GB of SRAM (1ns access time) but cost like 1GB of slow memory

Key concept:

Use a **hierarchy** of memory technologies:

- ❖ Small but fast memory near CPU
- ❖ Large but slow memory farther away from CPU

Memory Hierarchy



Cache Memory: The Basic Idea

- Keep the frequently and recently used data in **smaller but faster** memory
- Refer to bigger and slower memory:
 - Only when you cannot find data/instruction in the faster memory
- Why does it work?

Principle of Locality

Program accesses only a small portion of the memory address space within a small time interval

Cache: Types of Locality

■ Temporal locality

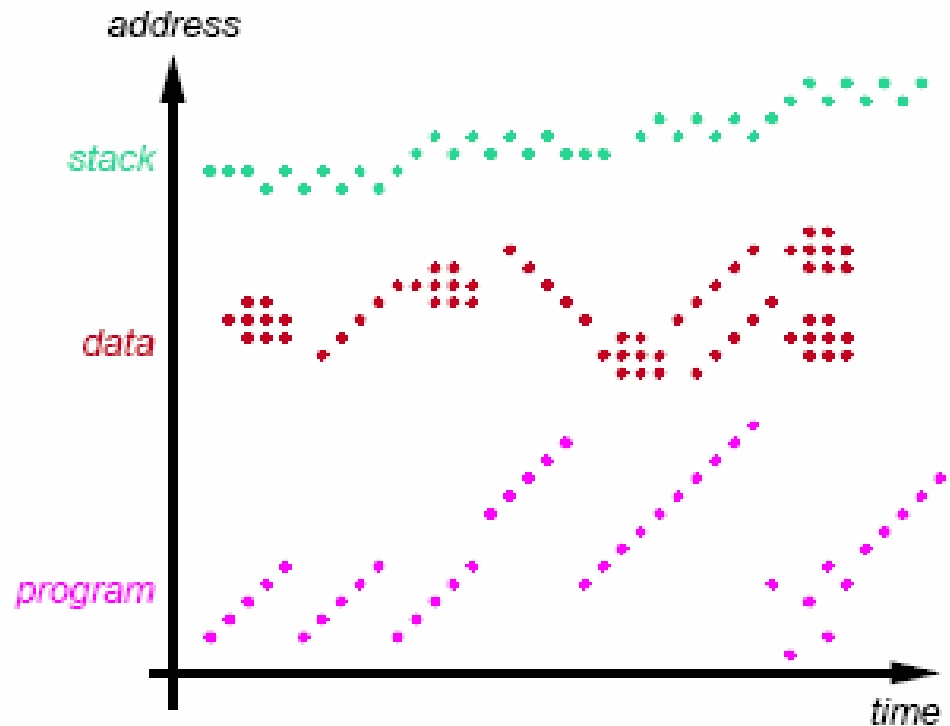
- If an item is referenced, it will tend to be referenced again soon

■ Spatial locality

- If an item is referenced, nearby items will tend to be referenced soon

■ Different locality for

- Instructions
- Data



Memory Access Time: Formula

$$\begin{aligned} &\text{Average Access Time} \\ &= \text{Hit rate} \times \text{Cache Memory Access Time} \\ &+ (1 - \text{Hit rate}) \times \text{Main memory access time} \end{aligned}$$

Example:

- Suppose our on-chip SRAM (cache) has **0.8 ns** access time, but the fastest DRAM (main memory) we can get has an access time of **10ns**. **How high a hit rate** do we need, to sustain an average access time of **1ns**?

Memory Access Time: Formula

Average Access Time

$$= \text{Hit rate} \times \text{Cache Memory Access Time} \\ + (1 - \text{Hit rate}) \times \text{Main memory access time}$$

Example:

- Suppose our on-chip SRAM (cache) has **0.8 ns** access time, but the fastest DRAM (main memory) we can get has an access time of **10ns**. **How high a hit rate** do we need, to sustain an average access time of **1ns**?

Let h be the desired hit rate.

$$1 = 0.8 \times h + (1 - h) \times 10$$

$$= 10 - 9.2h$$

$$9.82h = 9 \rightarrow h = 0.978$$

Hence we need a hit rate of **97.8%**.

Programming for Cache Performance

- Caches are not part of the programming interface
 - Programmer does not “know” about the cache
- Caches work best if you’re accessing the memory sequentially
 - Use a sequential data structure such as an array (e.g., Database or Scientific codes)
 - Use smaller data types and organize the data so there aren't any alignment holes
 - Improve **spatial locality** so that each cache line is used in full once it has been mapped to a cache
 - Improve **temporal locality** so that each cache line is used as many times as possible once it has been mapped to a cache

Loop Interchange

- Swap nested loops to access memory in sequential order inside the loop to improve cache locality

```
for (int j = 0; j < cols; ++j) {  
    for (int i = 0; i < rows; ++i) {  
        // Access array[i][j]  
    }  
}
```

```
for (int i = 0; i < rows; ++i) {  
    for (int j = 0; j < cols; ++j) {  
        // Access array[i][j]  
    }  
}
```

Loop Fusion

- Merging of loops can sometimes improve data locality

Cache “Blocking”

- Instead of accessing entire rows or columns, subdivide an array into blocks that conveniently fit into a portion of the L1 or L2 cache, and a thread can make repeated accesses to the data while the data is still in cache
 - For example, image processing and video applications are well suited to cache blocking because an image can be processed on smaller portions of the total image or video frame
 - Effectiveness of cache blocking depends on data block size, cache size, and the number of times the data is reused. An application can detect the data cache size using the `CPUID` instruction and dynamically adjust cache blocking tile sizes to maximize performance. As a general rule, cache block sizes should target approximately one-half to three-quarters the size of the physical cache
 - Cache blocking may require more memory accesses but improves locality of accesses
-

Cache “Blocking” Example

Function to perform matrix multiplication without cache blocking

```
def matrix_multiplication_without_blocking(A, B, C):
```

```
for i in range(N):
```

```
for j in range(N):
```

```
for k in range(N):
```

$$C[i][j] += A[i][k] * B[k][j]$$

Function to perform matrix multiplication with cache blocking

```
def matrix_multiplication_with_blocking(A, B, C):
```

```
for bi in range(0, N, block_size):
```

```
for bj in range(0, N, block_size):
```

```
for bk in range(0, N, block_size):
```

```
for i in range(bi, min(bi + block_size, N)):
```

```
for j in range(bj, min(bj + block_size, N)):
```

```
for k in range(bk, min(bk + block_size, N)):
```

$$C[i][j] += A[i][k] * B[k][j]$$

Cache “Blocking” Example

- The cache block size is set to 32. The goal is to read in a block of cache, do every bit of computing we can with the data in this cache, then load a new block of data into cache.
- There are 32 elements of A and 32 elements of B in cache at the same time and we try to do as much work with this data as we can before the increment to the next cache block.
- Blocking factors will be different for different architectures. Determine the blocking factors experimentally. For example, different blocking factors would be required for single precision versus double precision. Typically, the overall impact to performance can be significant.

Instruction-level Parallelism (ILP)

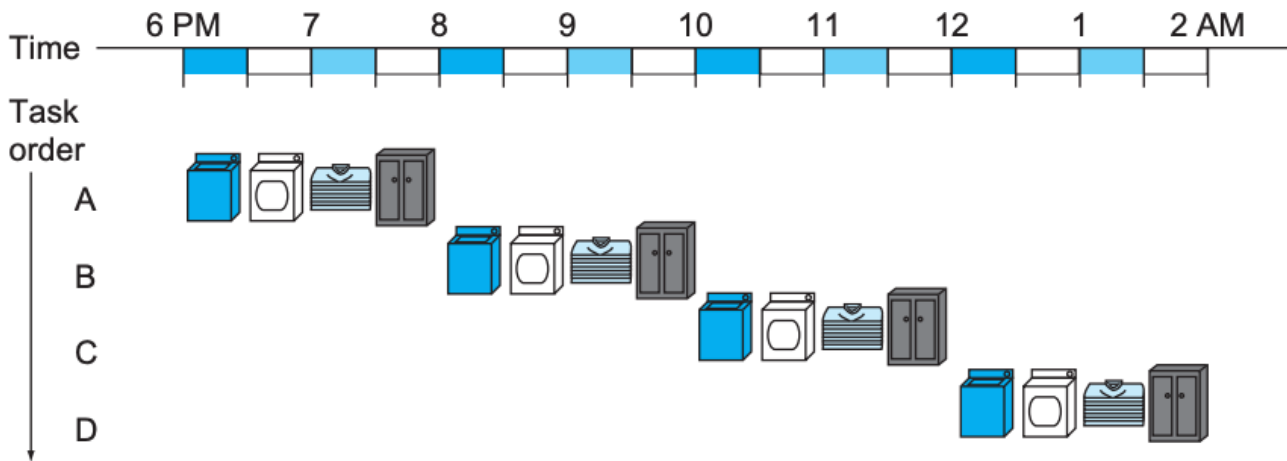
Instruction Level Parallelism

- **Instruction-Level Parallelism (ILP)**: processor overlaps the execution of instructions to improve performance
 - 2 approaches to exploit ILP:
 - 1) Rely on hardware to help discover and exploit the parallelism **dynamically** (e.g., GPU) , and
 - 2) Rely on software technology to find parallelism, **statically** at compile-time (e.g., data parallelism (SIMD), CUDA)
-

Assembly Line Processing

Strategies for improving performance

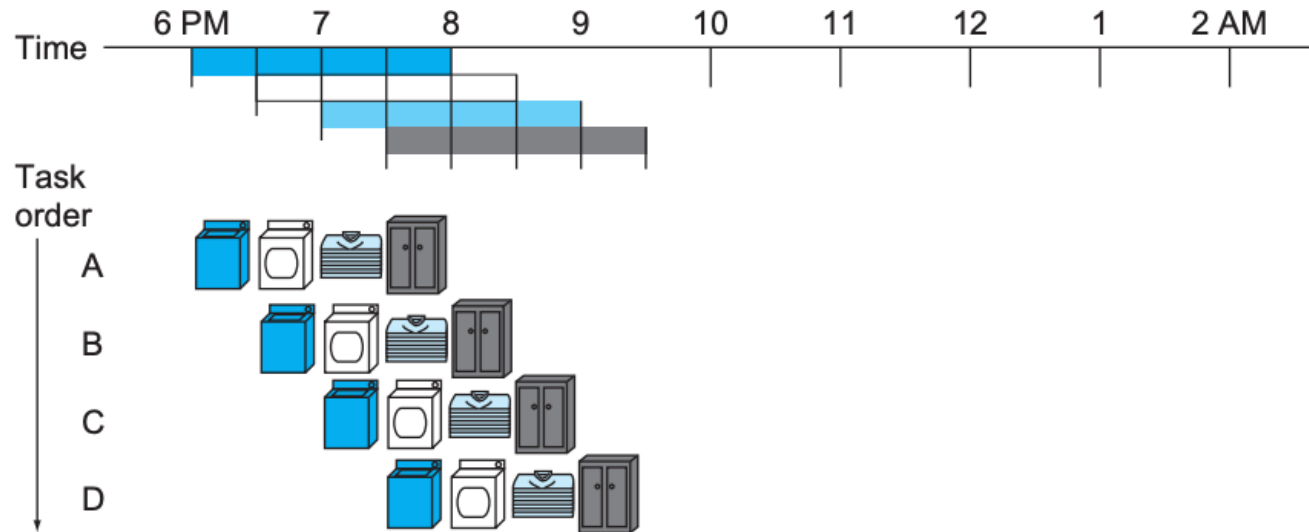
- 1 – Use multiple independent data paths accepting several instructions that are read out at once: *multiple-instruction-issue* or *superscalar*
- 2 – Overlap execution of several instructions, starting the next instruction before the previous one has run to completion: *(super)pipelined*



Source: Computer organization and design, Patterson & Hennessy, 5th Ed.2014

Pipelining in the laundry process

Pipelining

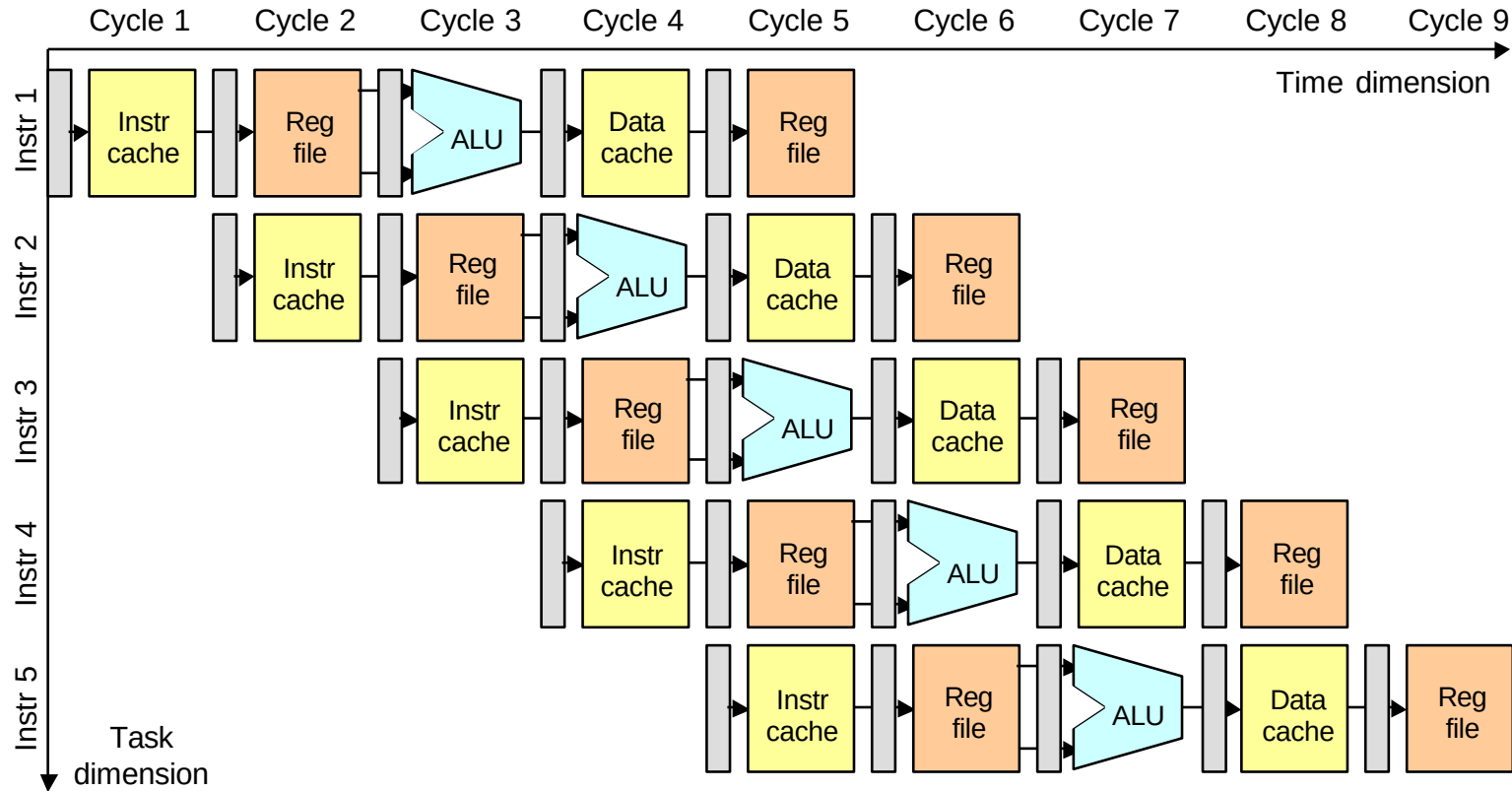


Pipelining

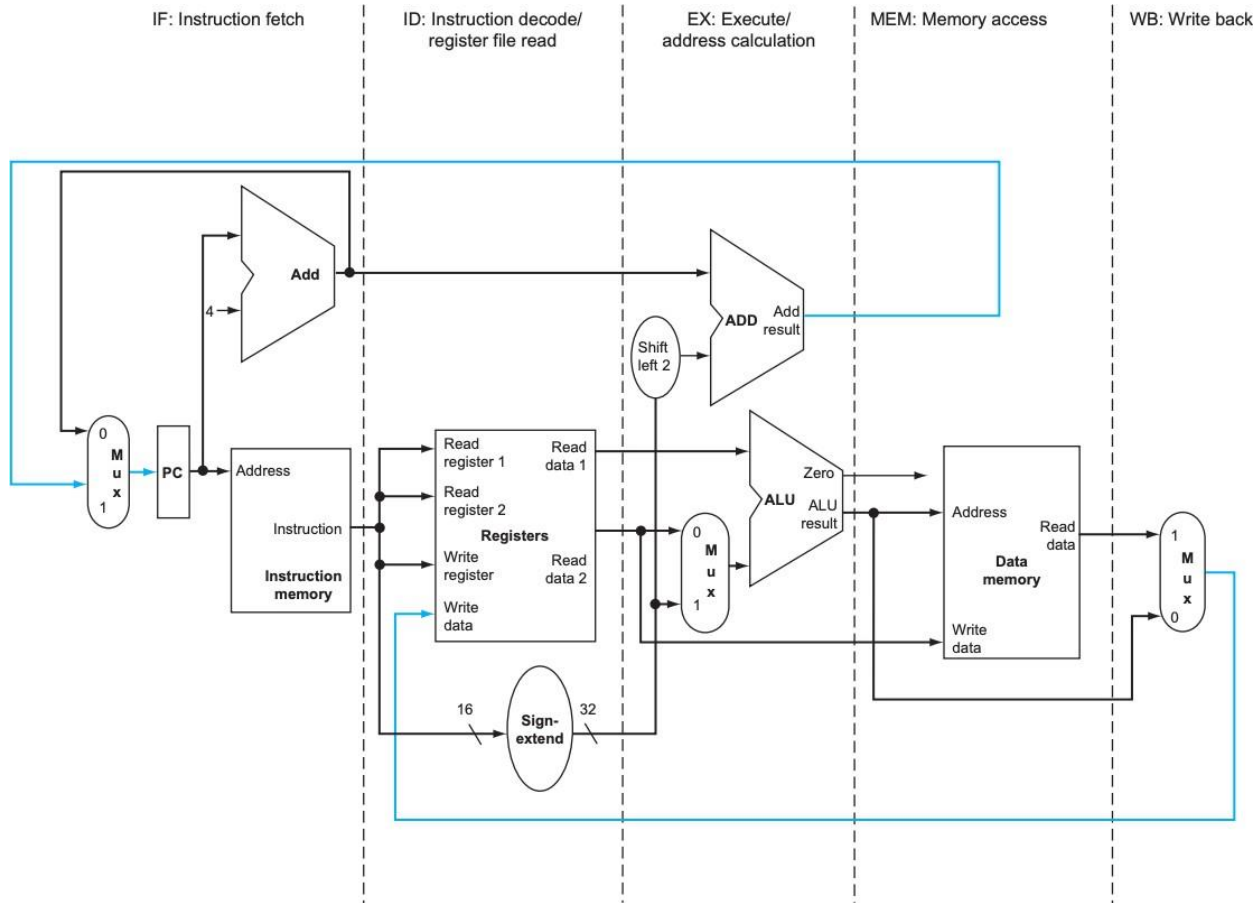
MIPS / RISC instructions classically take five steps:

- Fetch instructions from memory
- Read registers while decoding the instruction.
- Execute the operation or calculate an address.
- Access an operand in data memory.
- Write the result into a register.

Pipelining

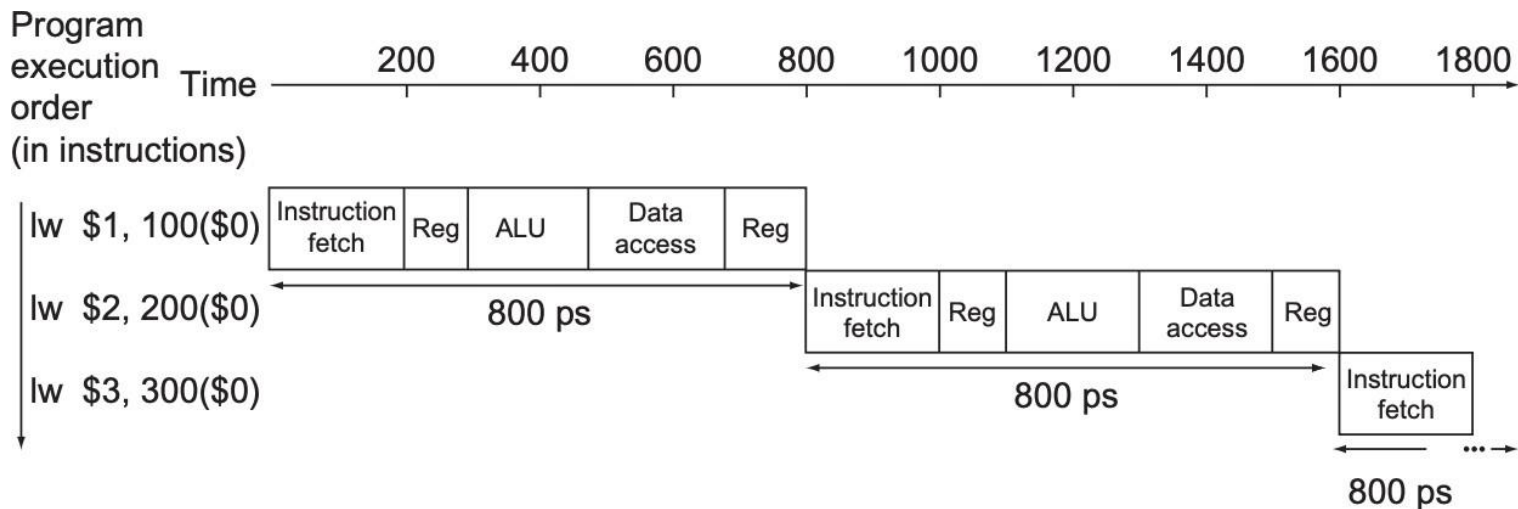


Pipelined Data Path Design

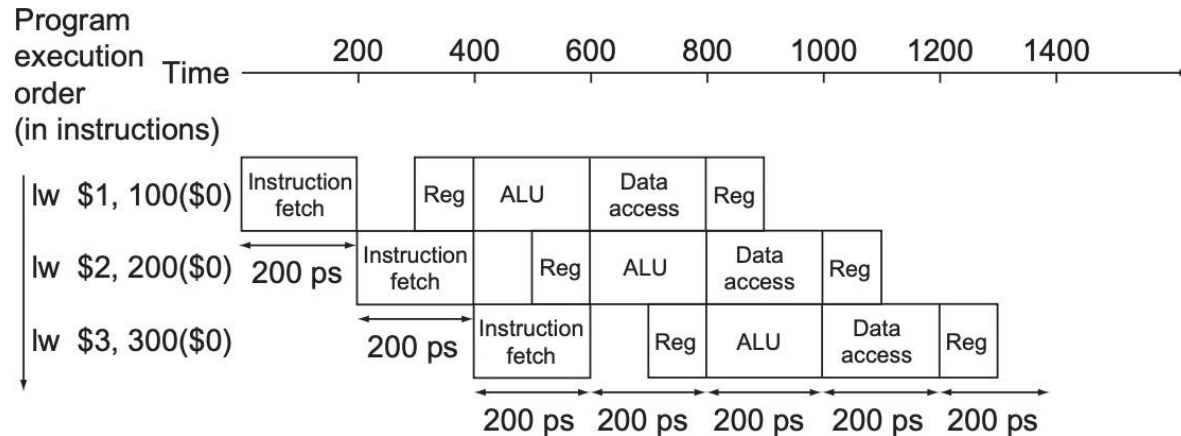
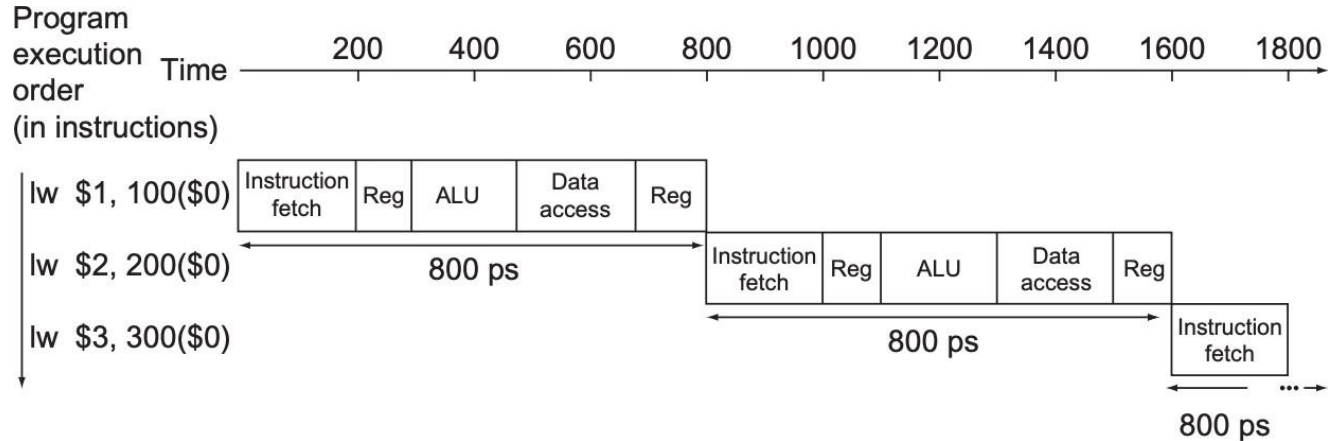


Speed Up with Pipelining

$$\text{Time between instructions}_{\text{pipelined}} = \frac{\text{Time between instructions}_{\text{nonpipelined}}}{\text{Number of pipe stages}}$$



Speed Up with Pipelining



Speed Up with Pipelining

Add 100,000 instructions to the previous 3.

Pipelined: $100,000 \times 200 + 1400 = 200,001,400$ ps

Non-pipelined: $100,000 \times 800 + 2400$ ps = $800,002,400$ ps

Ratio of total execution times for real programs on nonpipelined to pipelined processors is close to the ratio of times between instructions

$$\frac{800,002,400ps}{200,001,400ps} \simeq \frac{800ps}{200ps} \simeq 4.00$$

Limitations of Pipelining

In pipelining, there are instances where the subsequent instruction cannot be executed in the immediate clock cycle. These occurrences are termed as hazards, and they manifest in three distinct types.

- Structural hazard: hardware cannot support the combination of instructions that we want to execute in the same clock cycle.
- Data hazards: the pipeline would be stalled because one step must wait for another to complete.
- Control hazards: arises from the need to make a decision based on the results of one instruction while others are executing.

Multiple Issue

To achieve a CPI (Cycles Per Instruction) value of less than 1, it is necessary to execute multiple instructions within a single clock cycle. The solutions proposed to accomplish this include:

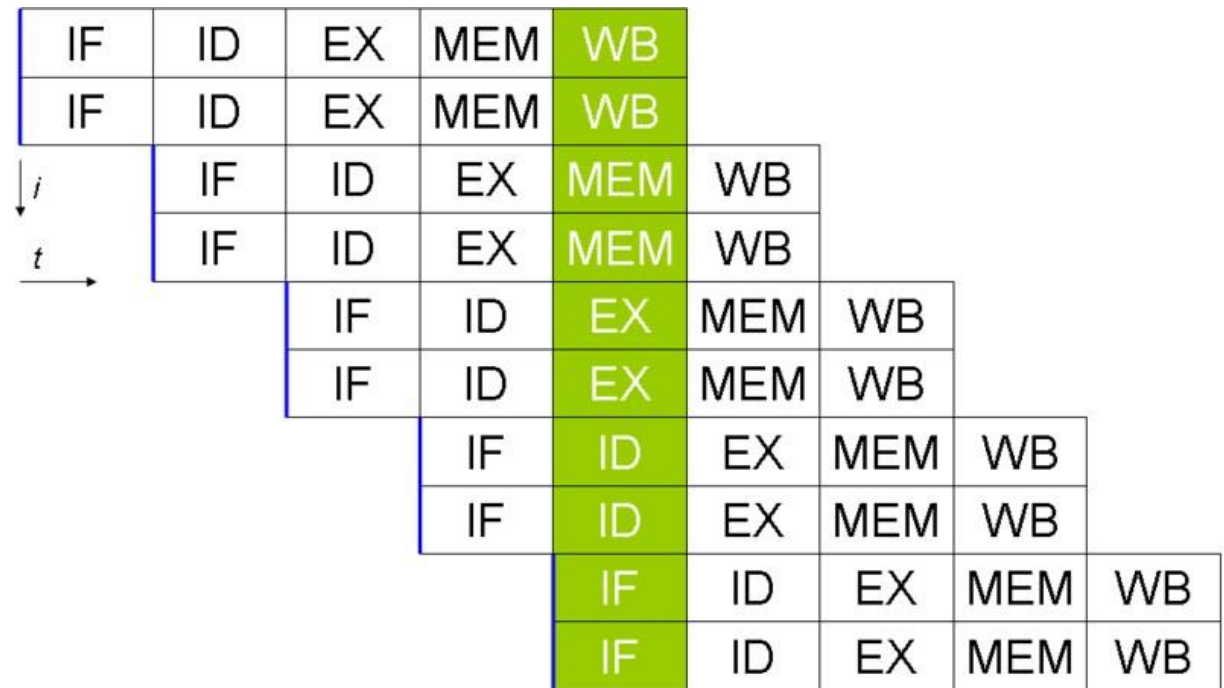
- Statically scheduled superscalar processors: These processors are designed to execute multiple instructions simultaneously by having multiple execution units.
- VLIW (Very Long Instruction Word) processors: VLIW processors execute multiple operations in parallel by packing them into a single long instruction word.
- Dynamically scheduled superscalar processors: These processors dynamically schedule instructions for execution based on data dependencies and available resources, allowing for efficient utilization of execution units.

Multiple Issue: Superscalar

■ Pipelining



► Superscalar



Multiple Issue: VLIW

- Package multiple operations into one instruction
- Example VLIW processor:
 - One integer instruction (or branch)
 - Two independent floating-point operations
 - Two independent memory references
- Must be enough parallelism in code to fill the available slots

Thread-level Parallelism (TLP)

Limits to ILP

- Conflicting studies of amount
 - Benchmarks (vectorized Fortran FP vs. integer C programs)
 - Hardware sophistication
 - Compiler sophistication
- How much ILP is available using existing mechanisms with increasing HW budgets?
- Do we need to invent new HW/SW mechanisms to keep on processor performance curve?
 - Intel MMX, SSE (Streaming SIMD Extensions): 64 bit ints
 - Intel SSE2: 128 bit, including 2 64-bit Fl. Pt. per clock
 - Motorola AltaVec: 128 bit ints and FPs
 - Supersparc Multimedia ops, etc.

Performance beyond ILP

- There can be much higher natural parallelism in some applications (e.g., Database or Scientific codes)
- Explicit **Multithreading** or **Data Level Parallelism**
- **Thread**: process with own instructions and data
 - thread may be a process part of a parallel program of multiple processes, or it may be an independent program
 - Each thread has all the state (instructions, data, PC, register state, and so on) necessary to allow it to execute
- **Data Level Parallelism**: Perform identical operations on data, and lots of data

Multithreading

- ILP exploits implicit parallel operations within a loop or straight-line code segment
- Use multiple threads of execution that are inherently parallel
- Goal: Use multiple instruction streams to improve
 1. Throughput of computers that run many programs
 2. Execution time of multi-threaded programs
- Multithreading could be more cost-effective to exploit than ILP

Multithreaded Execution

- Multithreading: multiple threads to share the functional units of 1 processor via overlapping
 - processor must duplicate independent state of each thread e.g., a separate copy of register file, a separate PC, and for running independent programs, a separate page table
 - memory shared through the virtual memory mechanisms, which already support multiple processes
 - HW for fast thread switch; much faster than full process switch ≈ 100 s to 1000 s of clocks
- When to switch?
 - Alternate instruction per thread (fine grain)
 - When a thread is stalled, perhaps for a cache miss, another thread can be executed (coarse grain)

Fine-Grained Multithreading

- Switches between threads on each instruction, causing the execution of multiples threads to be interleaved
- Usually done in a round-robin fashion, skipping any stalled threads
- CPU must be able to switch threads every clock
- Advantage is it can hide both short and long stalls, since instructions from other threads executed when one thread stalls
- Disadvantage is it slows down execution of individual threads, since a thread ready to execute without stalls will be delayed by instructions from other threads
- Used on Sun's Niagara

Coarse-Grained Multithreading

- Switches threads only on costly stalls, such as L2 cache misses
- Advantages
 - Relieves need to have very fast thread-switching
 - Doesn't slow down thread, since instructions from other threads issued only when the thread encounters a costly stall
- Disadvantage is hard to overcome throughput losses from shorter stalls, due to pipeline start-up costs
 - Since CPU issues instructions from 1 thread, when a stall occurs, the pipeline must be emptied or frozen
 - New thread must fill pipeline before instructions can complete
- Because of this start-up overhead, coarse-grained multithreading is better for reducing penalty of high cost stalls, where pipeline refill $\ll$ stall time
- Used in IBM AS/400

Simultaneous Multi-threading

One thread, 8 units

1	Yellow							Yellow
2	Yellow	Yellow					Yellow	
3				Yellow	Yellow			
4								
5								
6	Yellow			Yellow		Yellow		
7		Yellow			Yellow			
8				Yellow				

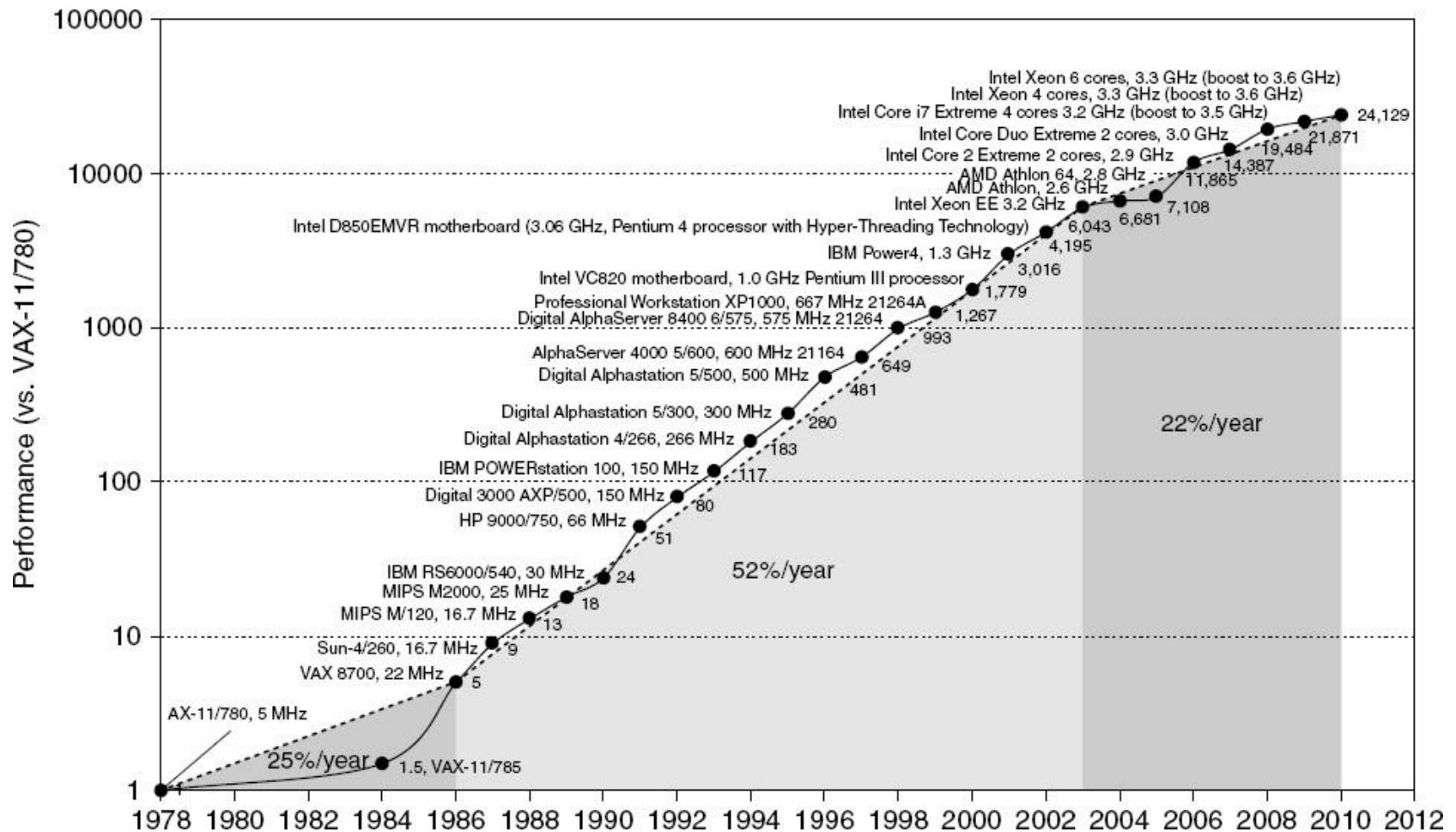
Two threads, 8 units

1	Yellow	Blue	Blue					Yellow
2	Yellow	Yellow	Blue			Blue	Yellow	
3	Blue			Yellow	Yellow			
4	Blue	Blue				Blue		
5		Blue						Blue
6	Yellow		Blue	Yellow	Blue	Yellow		
7		Yellow		Blue	Yellow	Blue		
8	Blue	Blue		Yellow		Blue		

Time (processor cycle)



Move to Multiprocessor



Multi-core Systems

- To speed up the execution of a program, we split it and distribute across many cores
- Faster or/and efficient
- Many ML tasks are suited for parallel processing, for example matrix multiplication

Multicore Objectives

- Use available transistors to add value
 - Provide better perf, perf/cost, perf/watt
- Effectively share expensive resources
 - Socket/pins:
 - DRAM interface
 - Coherence interface
 - I/O interface
 - On-chip area/power
 - Mem controller
 - Cache
 - FPU? (Conjoined cores, e.g. Niagara)

Multi-core Benefits

- ❖ For a 2 core system, gain 1.7X the performance without increasing the original power consumption
 - ❖ They improve an operating system's ability to multitask applications
 - ❖ Another benefit comes from individual applications optimized for multi-core processors
-

Homogeneous Cores

- ❖ Homogenous cores are all exactly the same: same frequencies, cache sizes, functions
- ❖ Easier to produce since the same instruction set is used across all cores and each core contains the same hardware
- ❖ Most designs targeting desktops, laptops and servers are homogeneous

Heterogeneous Cores

- ❖ Each core in a heterogeneous system may have a different function, frequency, memory model
- ❖ Each core in a heterogeneous environment run its own specialized instruction set. This model is more complex, but may have efficiency, power, and thermal benefits that outweigh its complexity
- ❖ In the embedded sphere, heterogeneity is more common, evidenced by for instance the Cell processor and the typical architecture of mobile phones
- ❖ For heterogeneous systems, programming tools remain a challenge as compared to on a homogeneous system

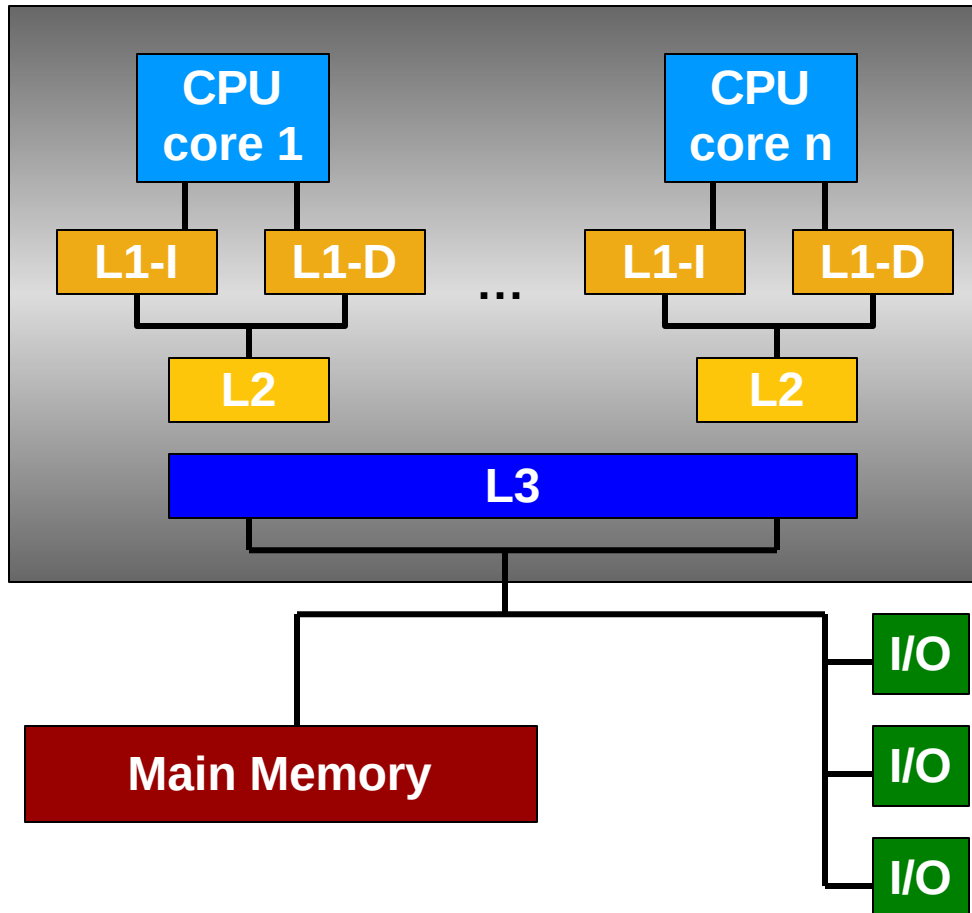
Heterogeneous Cores

- Heterogeneous cores
 - E.g., GP cores, GPUs, cryptography, vector cores, floating point coprocessors
 - Heterogeneous cores may be programmed differently
 - Mechanisms must exist to transfer activity from one core to another
 - Fine-grained: e.g. FP co-processor, use ISA
 - Coarse-grained: transfer computation using APIs
- Examples:
 - Cores with different ISAs
 - Cores with different cache sizes, different issue width, different branch predictors
 - Cores with different micro-architectures (in-order vs. out-of-order)
 - Different types of cores (GP and SIMD)
- Goals:
 - Save area (more cores)
 - Save power by using cores with different power/performance characteristics for different phases of execution

Heterogeneous Cores

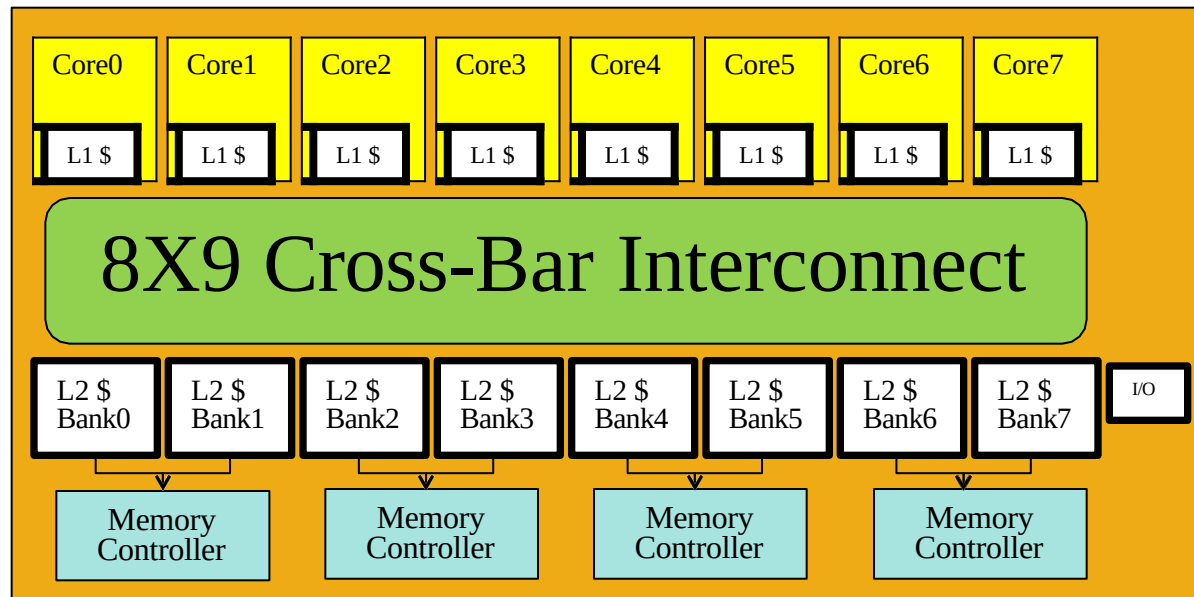
- Different applications may have better performance/power characteristics on some types of core (static)
- Same applications go through different phases that can use different cores more efficiently (dynamic)
 - Execution moves from core to core dynamically
 - Most interesting case (dynamic)
 - Cost of switching cores (must be infrequent: such as O/S time slice)
- Assume cores with same ISA but different performance/energy ratio
 - Need ability to track performance and energy to make decisions
 - Goal: minimize energy-delay product (EDP)
 - Periodically sample performance and energy spent
 - Run application on one or multiple cores in small intervals
 - Possible heuristics
 - Neighbor: pick one of the two neighbors at random, sample, switch if better
 - Random: select a core at random and sample, switch if better
 - All: sample all cores and select the best
 - Consider the overhead of sampling

Real-Life Example: Intel Core i7

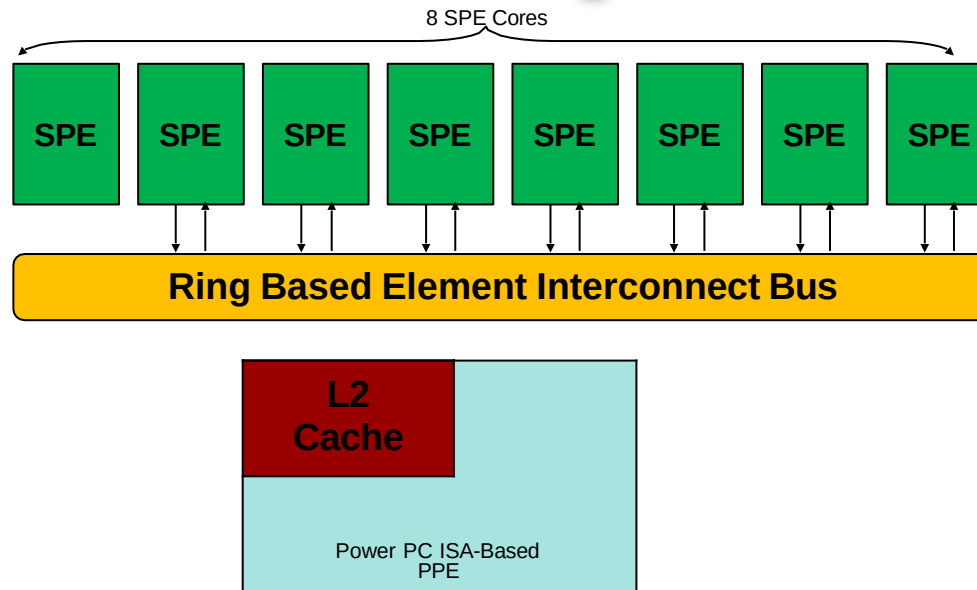


Shared L3 Cache

Real-life Example: IBM Power 4/5/6/7)



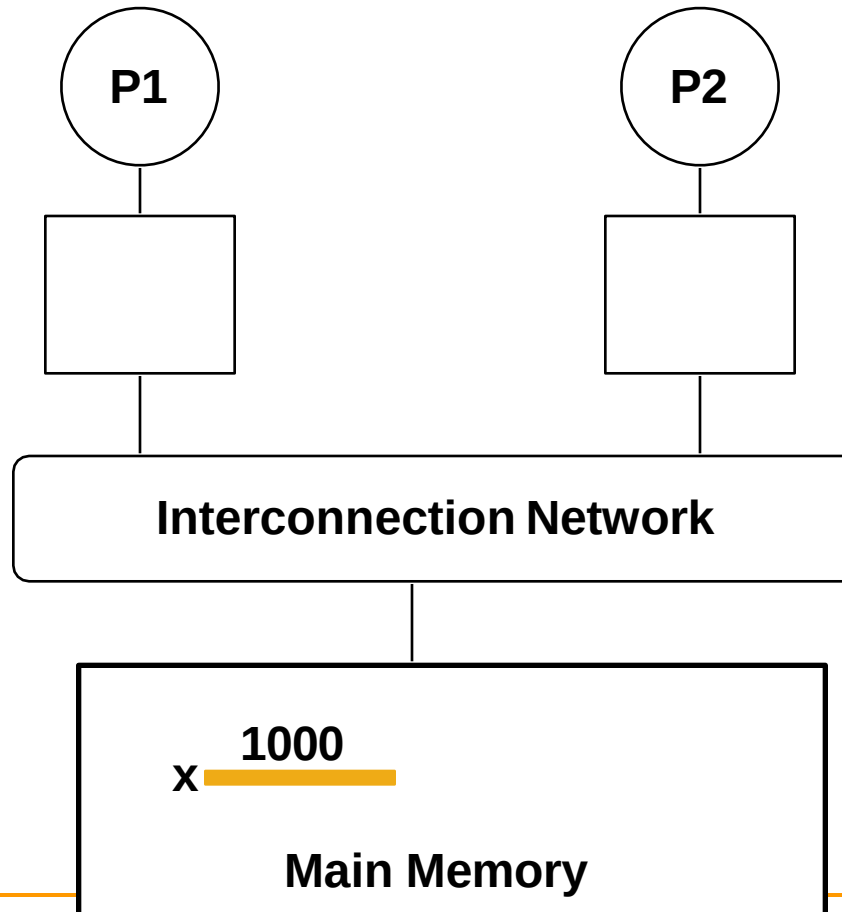
Real-life Example: IBM CELL



- ONE PowerPC processing element (PPE)
 - 2-WAY SMT Power CORE
- 8 Synergistic processing elements (SPEs)
 - SPE is 2-issue in-order processor
 - Two SIMD instructions can be issued in each cycle (vectors)
 - No coherence support between SPE and PPE (software-managed scratchpad memory in SPE)

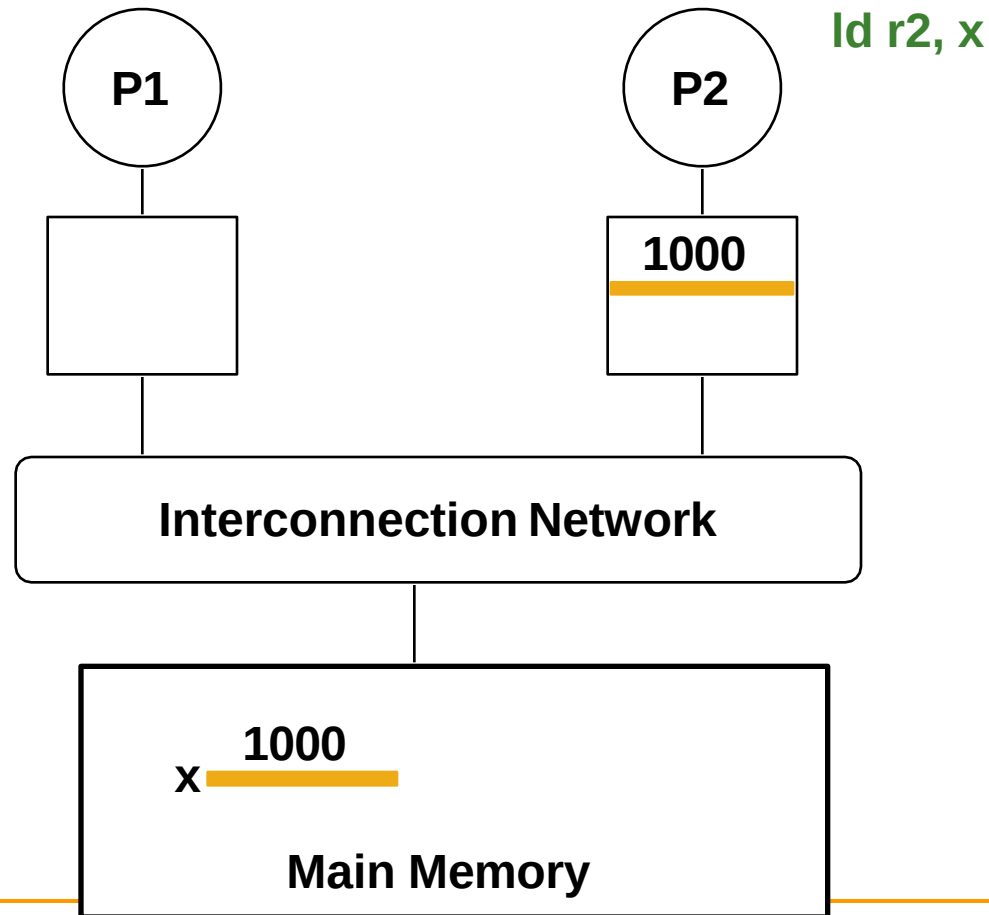
Cache Coherence Problem

- If multiple cores cache the same block, how do they ensure they all see a consistent state?



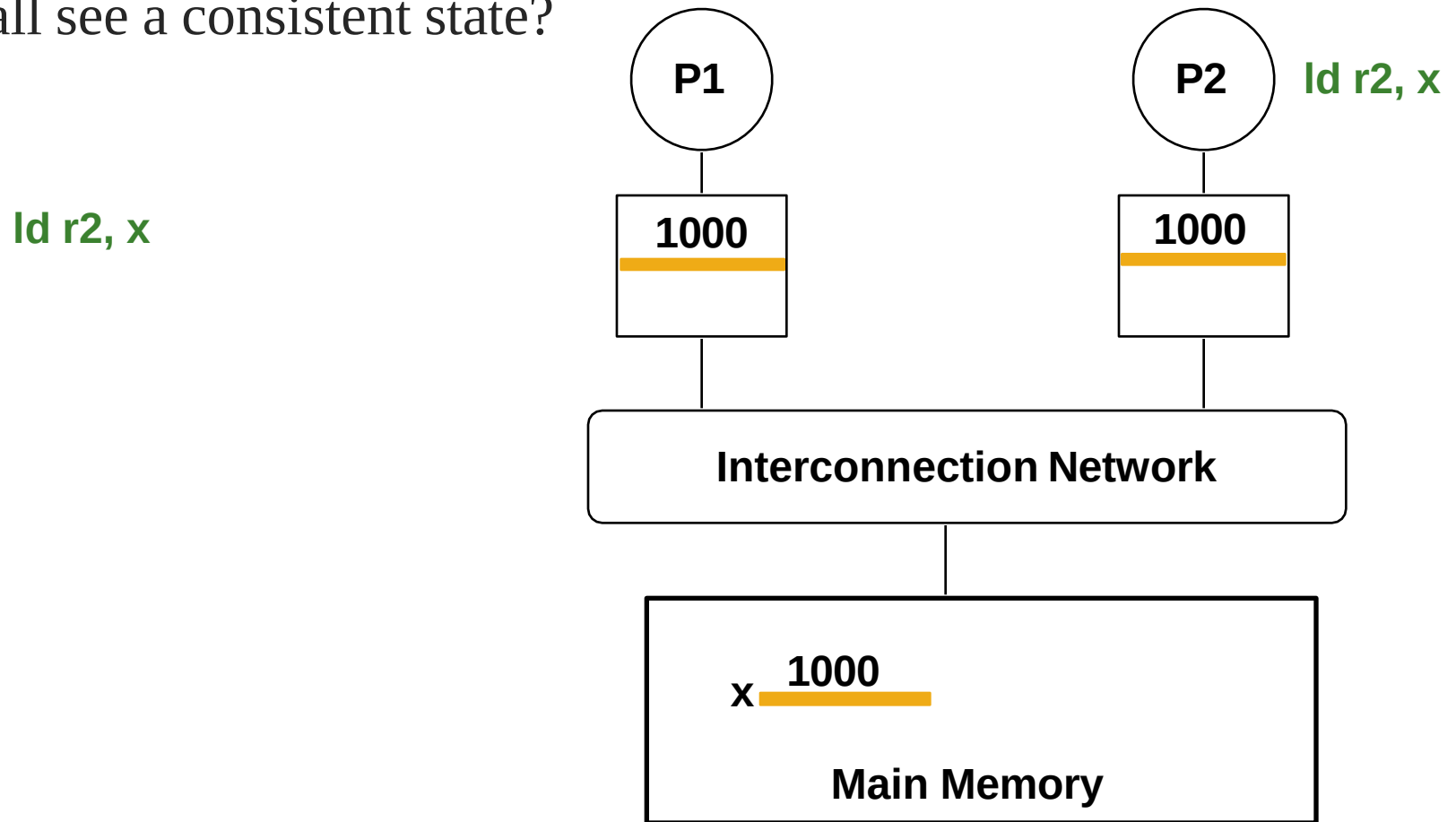
Cache Coherence Problem

- If multiple cores cache the same block, how do they ensure they all see a consistent state?



Cache Coherence Problem

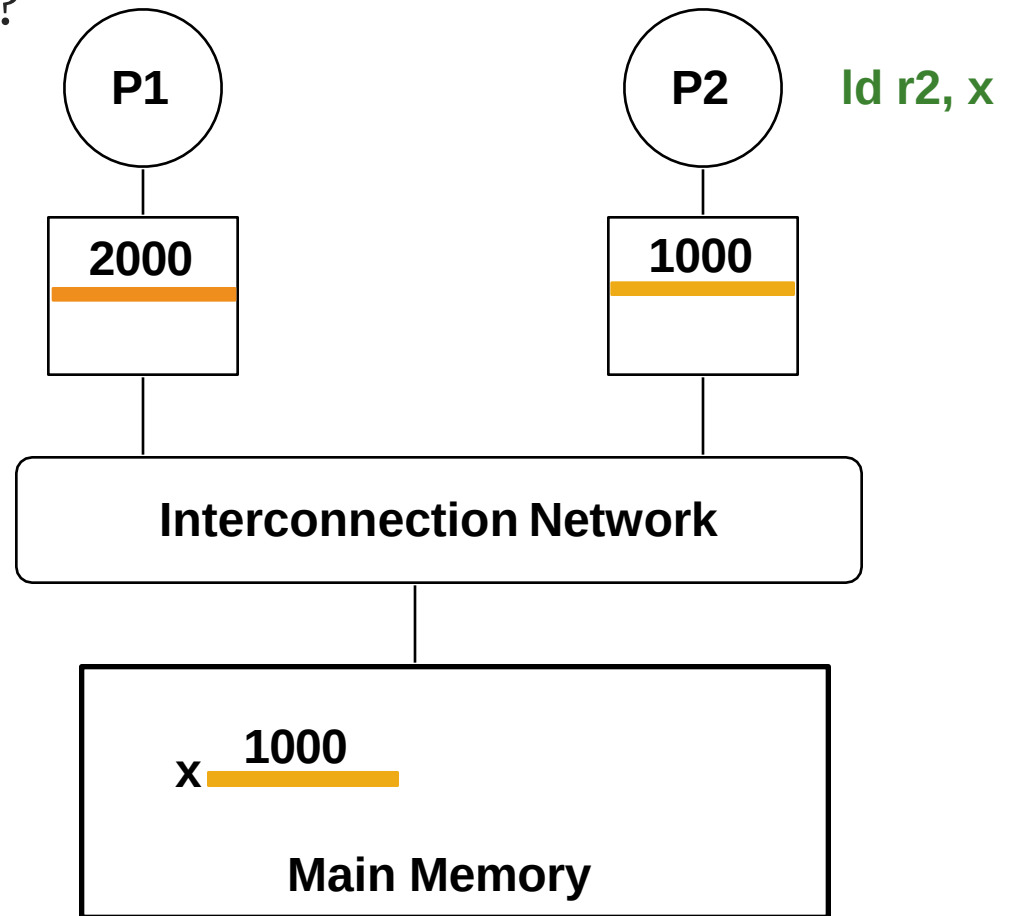
- If multiple cores cache the same block, how do they ensure they all see a consistent state?



Cache Coherence Problem

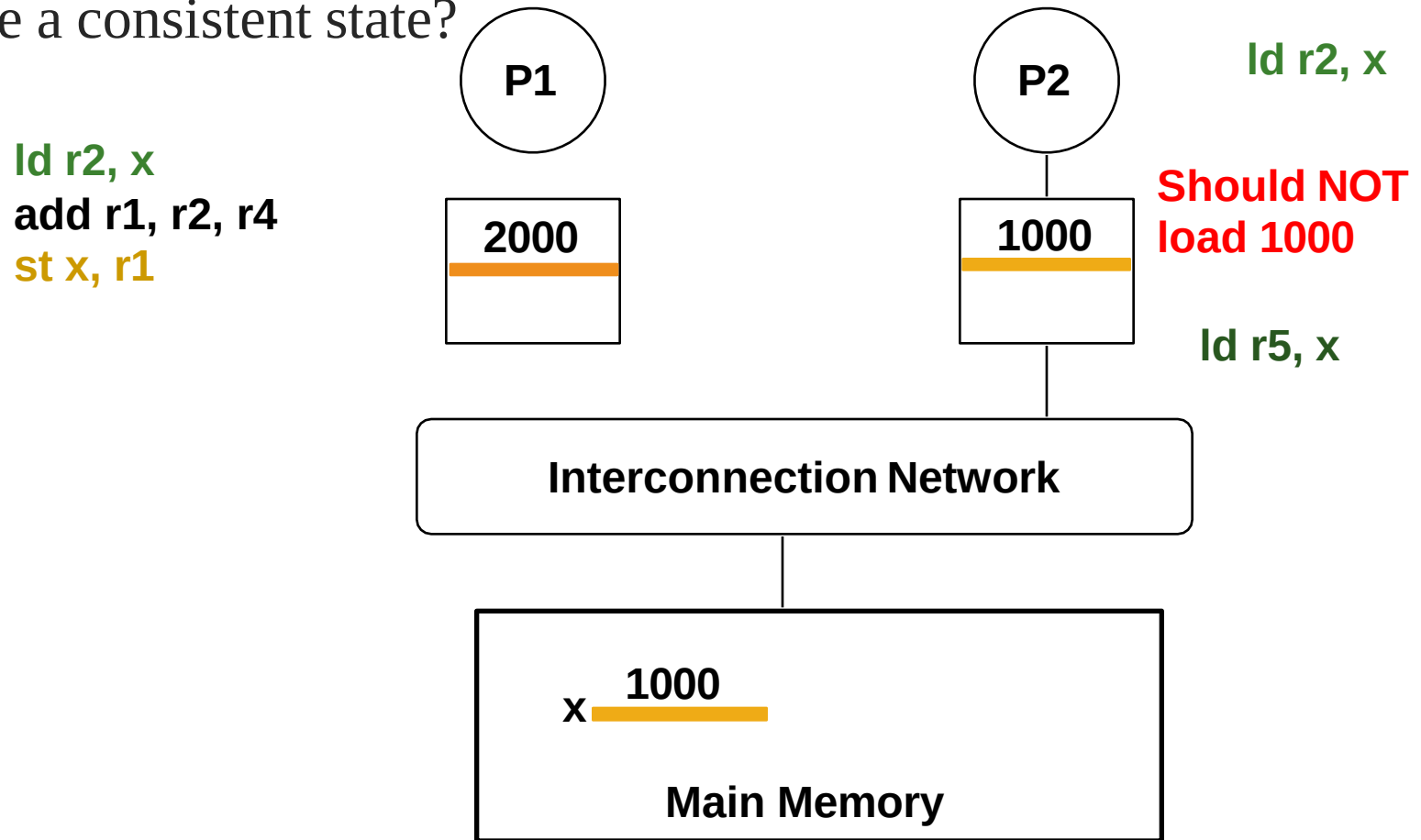
- If multiple cores cache the same block, how do they ensure they all see a consistent state?

ld r2, x
add r1, r2, r4
st x, r1



Cache Coherence Problem

- If multiple cores cache the same block, how do they ensure they all see a consistent state?



Load Balancing Problem

- **Round-robin scheduling:** Incoming requests are distributed among the available worker processes or threads in a cyclic order.
 - **Dynamic adjustment:** The system monitors the workload of each worker and adjusts the distribution of incoming requests accordingly.
 - **Health monitoring:** The load balancer continuously checks the health and availability of each worker.
-

Virtual Memory and Memory-Mapped Files

Virtual memory

- Main memory can act as a “cache” for the secondary storage.
 - This is called virtual memory.
 - There were two major motivations for virtual memory:
 - to allow efficient and safe sharing of memory among multiple programs, such as for the memory needed by multiple virtual machines for cloud computing,
 - and to remove the programming burdens of a small, limited amount of main memory.
-

Virtual memory

- For multiple virtual machines to utilize shared memory, it's crucial to safeguard each virtual machine from interference by others.
 - This ensures that each program can exclusively access and modify the specific portions of main memory allocated to it.
 - Main memory should only hold the active segments of numerous virtual machines, akin to how a cache stores solely the active part of a single program.
 - Thus, the principle of locality enables virtual memory as well as caches, and virtual memory allows us to efficiently share the processor as well as the main memory.
-

Virtual Memory

- The virtual machines sharing the memory change dynamically while the virtual machines are running.
 - Because of this dynamic interaction, we would like to compile each program into its own *address space*
 - a separate range of memory locations accessible only to this program.
-

Virtual Memory

- Virtual memory implements the translation of a program's address space to physical addresses.
 - This translation process enforces protection of a program's address space from other virtual machines.
 - The second motivation for virtual memory is to allow a single user program to exceed the size of primary memory.
-

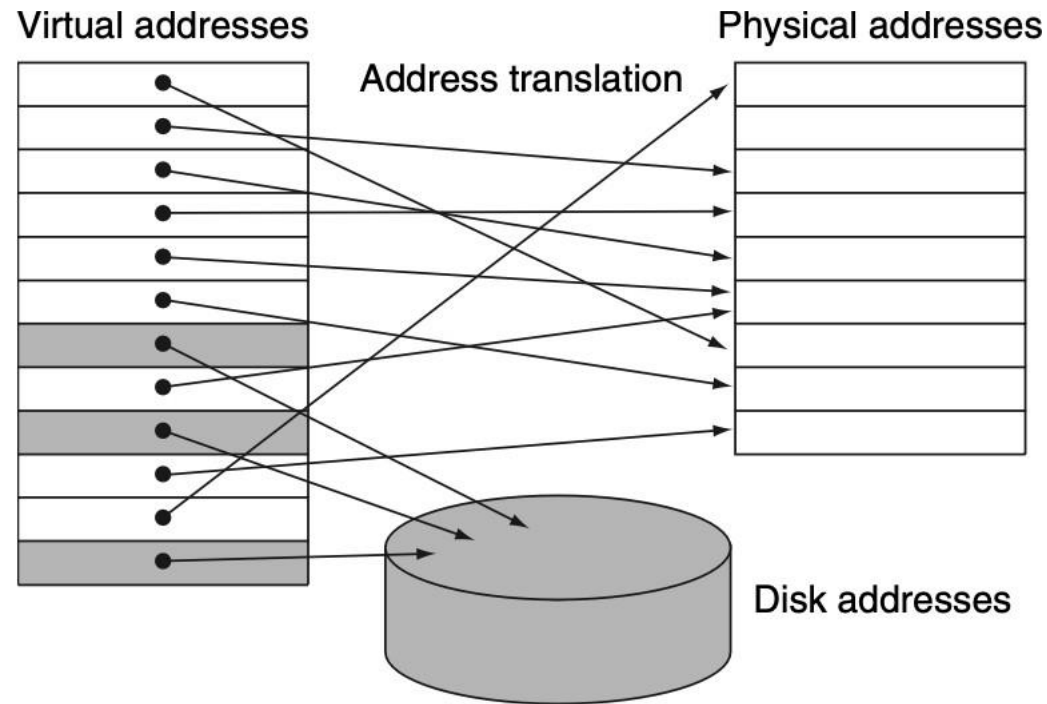
Virtual Memory

- The second motivation for virtual memory is to allow a single user program to exceed the size of primary memory.
 - Formerly, if a program became too large for memory, it was up to the programmer to make it fit.
 - Programmers divided programs into pieces and then identified the pieces that were mutually exclusive.
 - Virtual memory, which was invented to relieve programmers of this difficulty, automatically manages the two levels of the memory hierarchy represented by main memory (sometimes called physical memory to distinguish it from virtual memory) and secondary storage.
-

Virtual Address

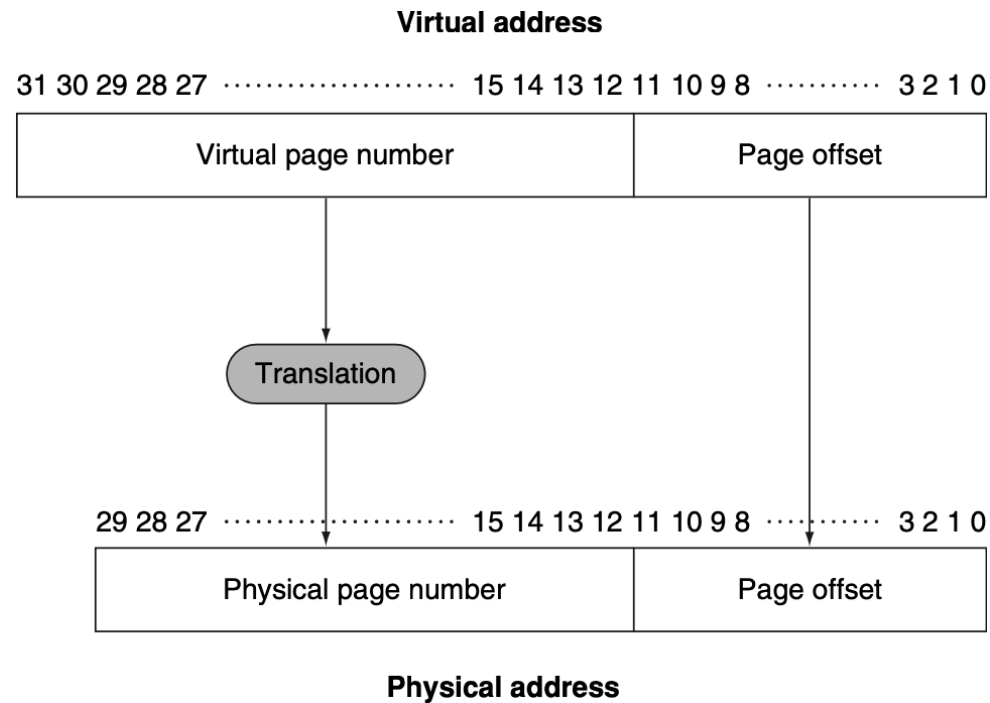
- A virtual memory block is called a *page*, and
 - a virtual memory miss is called a *page fault*.
 - With virtual memory, the processor produces a virtual address,
 - which is translated by a combination of hardware and software to a physical address, which in turn can be used to access main memory.
-

Address Translation

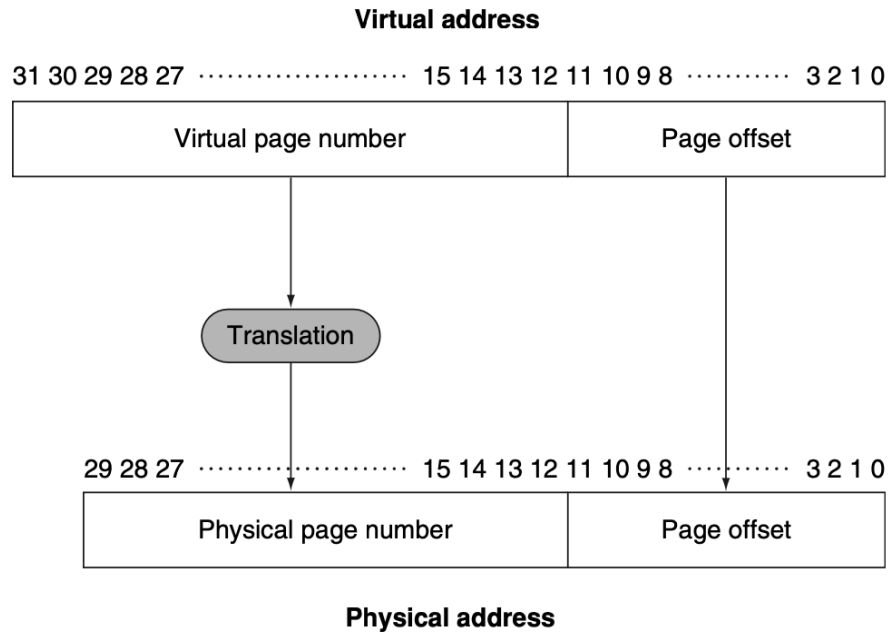


Virtual Memory

- In virtual memory, the address is broken into a virtual page and a page offset.



Virtual Memory

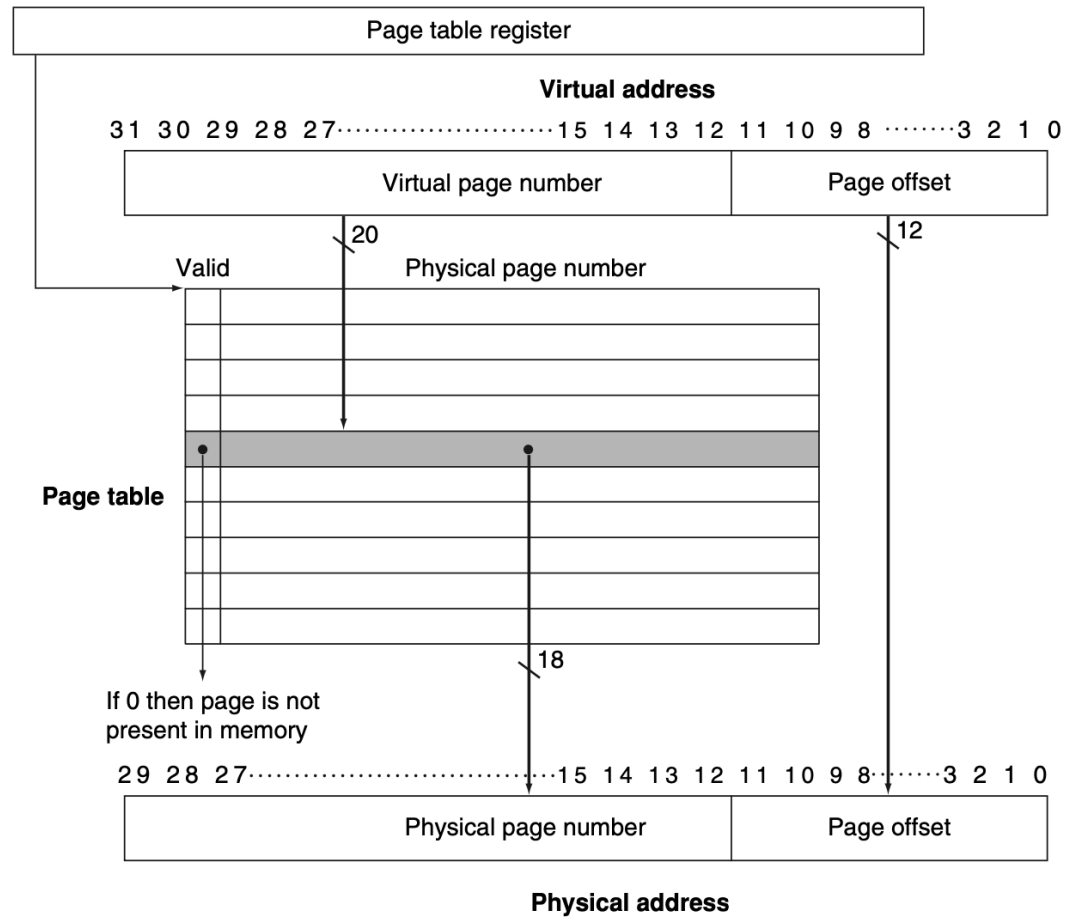


- The page size is $2^{12} = 4Kb$
- The number of physical pages allowed in memory is 2^{18}
- Main memory can have 1 GB
- Virtual address space is 4Gb

Page Table

- The table containing the virtual to physical address translations in a virtual memory system.
 - The page table, together with the program counter and the registers, specifies the state of a virtual machine.
 - Rather than save the entire page table, the operating system simply loads the page table register to point to the page table of the process it wants to make active.
 - Each process has its own page table, since different processes use the same virtual addresses.
-

Page Table



Page Fault

- If the valid bit for a virtual page is off , a page fault occurs.
 - Once the operating system gets control, it must find the page in the next level of the hierarchy (usually flash memory or magnetic disk) and decide where to place the requested page in main memory.
 - Because we do not know ahead of time when a page in memory will be replaced, the OS usually creates the space on flash memory or disk for all the pages of a process when it creates the process.
 - This space is called the swap space.
-

Page Fault

- The operating system also creates a data structure that tracks which processes and which virtual addresses use each physical page.
 - When a page fault occurs, if all the pages in main memory are in use, the operating system must choose a page to replace.
 - Because we want to minimize the number of page faults, most operating systems try to choose a page that they hypothesize will not be needed in the near future.
 - Using the past to predict the future, operating systems follow the least recently used (LRU) replacement scheme
-

Page Fault

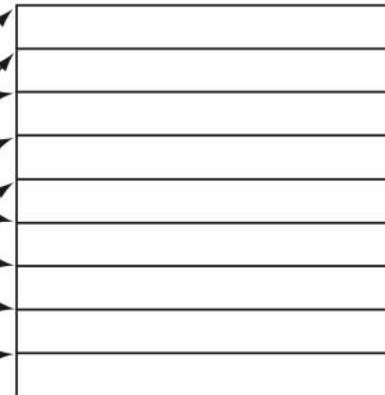
Virtual page
number

Page table

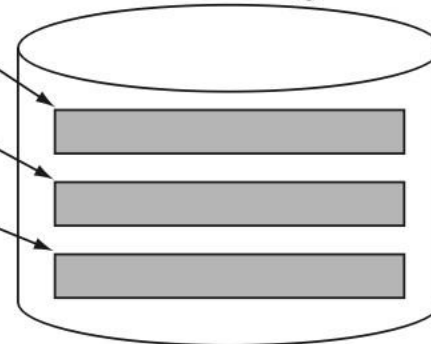
Valid Physical page or
disk address

1	•
1	•
1	•
1	•
0	•
1	•
1	•
0	•
1	•
1	•
0	•
1	•

Physical memory



Disk storage



Virtual Address

- With a 32-bit virtual address, 4 KB pages, and 4 bytes per page table entry, we can compute the total page table size

$$\text{Number of page table entries} = \frac{2^{32}}{2^{12}} = 2^{20}$$

$$\text{Size of page table} = 2^{20} \text{ page table entries} \times 2^2 \frac{\text{bytes}}{\text{page table entry}} = 4MB$$

Making Address Translation Fast

- Modern processors include a special cache that keeps track of recently used translations.
- It is called translation-lookaside buffer (TLB)

